

AutoTraQ Cheat Sheet — Fatima (Charts & Data Visualization)

Your Section: Charts / Data Visualization (Recharts + React)

Quick Overview

AutoTraQ's data visualization layer uses **Recharts** — a React-native charting library built on top of D3.js. The charts live primarily on the **Dashboard page** and provide real-time visual insights into inventory health, trends, and analytics.

Tech Stack

- **Recharts** — React charting library (wraps D3.js in React components)
 - **React** — component framework
 - **TypeScript** — typed JavaScript
 - **Lucide React** — icons used alongside charts for KPI cards
-

Where the Charts Live

```
frontend/src/pages/DashboardPage.tsx    ← Main dashboard with all charts
frontend/src/components/SearchStatsCards.tsx ← Stats cards for search results
frontend/src/components/Skeleton.tsx     ← Loading states for charts
```

Dashboard Components

The Dashboard (`DashboarPage.tsx`) has these visual sections:

1. KPI Cards (Key Performance Indicators) - Total Parts count - Total Inventory units - Total Inventory Value (dollars) - Pending Requests count - Low Stock Alerts count Each card has an icon (Lucide), a number, and color coding.

2. Inventory History Line Chart

```
<ResponsiveContainer width="100%" height={300}>
  <LineChart data={history}>
```

```

    <CartesianGrid strokeDasharray="3 3" />
    <XAxis dataKey="date" />
    <YAxis />
    <Tooltip />
    <Line type="monotone" dataKey="total" stroke="#6366f1" strokeWidth={2} />
  </LineChart>
</ResponsiveContainer>

```

- Shows inventory levels over time
- Uses `ResponsiveContainer` for fluid width
- `LineChart` with smooth curves (`type="monotone"`)
- Grid, axes, and tooltip for readability

3. Top Movers Section - Parts with the most activity (stock in/out events) - Shows part name, event count, and net change - Color-coded: green for net positive, red for net negative - Uses `TrendingUp` / `TrendingDown` icons

4. Dead Stock Analysis - Parts with no activity for extended periods - Shows part name, quantity sitting idle, days since last movement - Helps identify slow-moving inventory for clearance

5. Low Stock Alerts - Parts below their minimum stock threshold - Visual warning cards with current quantity vs. minimum

6. Recent Activity Feed - Latest inventory events (stock in/out) - Timeline-style with user, action, quantity, timestamp

How Recharts Works

Recharts uses **declarative, composable React components**:

```

import {
  LineChart, Line, XAxis, YAxis,
  CartesianGrid, Tooltip, ResponsiveContainer
} from 'recharts';

// Data is just an array of objects
const data = [
  { date: '2026-01-01', total: 150 },
  { date: '2026-01-15', total: 230 },
  { date: '2026-02-01', total: 195 },
];

// Chart is composed from building blocks
<ResponsiveContainer width="100%" height={300}>
  <LineChart data={data}>

```

```

    <CartesianGrid />          {/* Grid lines */}
    <XAxis dataKey="date" />    {/* X axis uses 'date' field */}
    <YAxis />                  {/* Y axis auto-scales */}
    <Tooltip />                {/* Hover tooltip */}
    <Line dataKey="total" />    {/* The actual line */}
  </LineChart>
</ResponsiveContainer>

```

Key Recharts components available: - `LineChart` / `Line` — trend lines (used for inventory history) - `BarChart` / `Bar` — comparisons - `PieChart` / `Pie` — proportions - `AreaChart` / `Area` — filled trend areas - `ResponsiveContainer` — makes charts fluid/responsive - `Tooltip` — hover info - `Legend` — chart legend - `CartesianGrid` — background grid

Data Flow for Charts

Backend API → frontend `api/client.ts` → `DashboardPage` state → Recharts components

1. `DashboardPage` calls the API on mount (`useEffect`)
 2. Data stored in React state (`useState`)
 3. While loading → `SkeletonChart` component shows placeholder
 4. Once loaded → Recharts renders the visualization
-

Skeleton Loading States

```

// from components/Skeleton.tsx
<SkeletonKPICards />    ← Placeholder for KPI cards while loading
<SkeletonChart />       ← Placeholder for charts while loading
<SkeletonList />        ← Placeholder for lists while loading

```

These provide a smooth UX instead of blank screens or spinners.

Talking Points for the Meeting

- "We chose **Recharts** for data visualization because it's built on **D3.js** but provides a much more **React-friendly API** — declarative components instead of imperative DOM manipulation"
- "The **Dashboard** serves as the central intelligence hub — at a glance you can see inventory health, trends, alerts, and activity"
- "The **Inventory History chart** tracks stock levels over time using a responsive line chart — managers can spot trends and seasonal patterns"

- "**Top Movers** analysis identifies the highest-activity parts — useful for demand forecasting and reorder planning"
- "**Dead Stock analysis** flags parts sitting idle — this directly impacts working capital and warehouse efficiency"
- "We implemented **skeleton loading states** so charts don't flash or jump — the UI feels polished even on slow connections"
- "All charts use `ResponsiveContainer` so they're **fully responsive** — works on desktop, tablet, and mobile without separate layouts"
- "The chart data comes from the same REST API the rest of the app uses — **single source of truth**, no separate analytics backend needed"